Quality Code 0.28

The commands, and what is new. From Jira ticket to tests you can trust.

What is new since 0.16

- `/quality-code:ci` (0.24, 0.25): the testing phases for GitHub Actions and GitLab CI. It sees whether the branch changed backend or frontend and runs only the touched tests, per module. The generated CI file runs every test of the test folders (`--changed` for the change-only variant).
- `/quality-code:routine` (0.26): a check on a schedule, run by Claude's scheduler. It fetches what others pushed, runs a task or the whole suite, says when green turned red (with the commits in between), and can run every test several times to track stability.
- **MCP server in Python** (0.19 to 0.23): every command is a tool. Local over stdio (no sign-in), or remote over HTTP with Entra ID sign-in and a Docker image. Proven end to end with a mock tenant.
- **Pilot-report fixes** (0.28): tests that sit beside the code are found; the expected-value check reads only the outcome of a one-line Given/When/Then; red verdicts give the right reason for the ticket type; a failing UI test shows the line, locator and page; screens are checked against the app's routes.
- **Testcontainers** (0.27): setup adds it to Vitest, pytest, Go and .NET backends, so integration tests run on real databases.

Update first

In Claude Code (CLI or the desktop Code tab):

```
/plugin update quality-code@teamsystem
/reload-plugins
```

Type `/quality-code:` and check that the list shows run-pipeline, spec, ci and routine.

1. Set up (once)

Command	What it does	Example
/quality-code:setup	Connects source code (GitLab / GitHub / both / none), Jira (required) and Figma (optional), finds the project, its stack, Jira project and base branch, shows one card to confirm, writes .claude/quality-code.json. Option 2 is a personal setup for QA and PO with no repo.	`/quality-code:setup`
/quality-code:setup advanced	Feature labels, QA plans, title prefixes, templates and redaction.	`/quality-code:setup advanced`
/quality-code:config	Shows and changes one setting at a time: the project config, your plugin settings (Jira email, GitLab, Xray) and which tokens are set. Every change is checked before it is saved.	`/quality-code:config`

2. Every day: from ticket to tests

Command	What it does	Example
/quality-code:spec	The main command. Takes the ticket of your branch (or a key), runs the readiness gate, shows the plan, writes the tests in your test folders, runs them through the pipeline and reads the card back. It asks three times at most.	`/quality-code:spec` `/quality-code:spec ABC-123`
/quality-code:spec KEY --gate-only	Only the readiness score of a ticket, no tests.	`/quality-code:spec ABC-123 --gate-only`
/quality-code:start, :write	Old names, kept: the same as spec (start = no key, write = with a key).	`/quality-code:write ABC-123`
/quality-code:ticket-eval	Scores one ticket out of 100 (70 passes). Refused: fill it in here, post a comment on Jira, or continue anyway (marked RISK).	`/quality-code:ticket-eval ABC-123`
/quality-code:ticket	Finds a ticket and shows it in short, with tips for its author. Takes a key or filters.	`/quality-code:ticket assignee:me status:"In Progress"`

3. Run and grade the tests

Command	What it runs	Example
/quality-code:run-frontend	Every Playwright spec of the project.	`/quality-code:run-frontend`
/quality-code:run-backend	Every backend spec: Vitest, pytest, JUnit, Go or .NET, whichever the project uses.	`/quality-code:run-backend`
/quality-code:run-pipeline	Both together, the full suite. --branch: only the tests your branch changed. Spec paths: only those. --evals: the frozen evals.	`/quality-code:run-pipeline --branch`

All three take `--key KEY` (judge one ticket's tests) and `--flaky N` (run N times, catch flaky tests). The card ends in one verdict, decided by a script, never by the model:

Verdict	Meaning
green	The tests run, assert the ticket's values and catch changes to them.
tests red	The checks pass and some tests fail: a real bug, or a fix not merged yet.
checks failed	A problem with the tests themselves: hollow, a missing ticket value, a removed assertion.
not verified	Something could not run (the app is down, no type-check, no runner). Never counted as a pass.

4. CI and scheduled checks

Command	What it does	Example
/quality-code:ci	For CI (GitHub, GitLab) or a terminal. Detects whether the branch touched backend or frontend and runs only the added or changed tests per module; everything on the default branch or with --all. Exit 0 green, 1 failed, 2 usage, 3 not verified. Writes JUnit, a GitHub step summary and annotations.	`/quality-code:ci` `/quality-code:ci --all`
/quality-code:ci --init github|gitlab	Writes the CI file for you (never overwrites). It runs every test of the test folders; add --changed to run only what the branch touched.	`/quality-code:ci --init github --write`
/quality-code:routine run	Creates a schedule with Claude's scheduler: once or on a period. Fetches the latest commits, runs a task (--task KEY) or the whole suite, and reports a change from green to red with the commits in between.	`/quality-code:routine run --task ABC-123 --since-last`
/quality-code:routine status	Lists the routines, the last result and the history; stops a routine.	`/quality-code:routine status`
--stability N	Runs every test N times and gives a verdict per test: STABLE, UNSTABLE or NOT ENOUGH DATA, with the pass rate over time. Alias --flaky.	`/quality-code:routine run --stability 5`

5. For QA and product owners

Command	What it does	Example
/quality-code:tickets-eval	Scores a whole Jira project, read-only: one line per ticket, how many pass, what is missing most.	`/quality-code:tickets-eval ABC --user me`
/quality-code:xray	Writes XRay test cases from a ticket: a CSV and a case.xlsx per ticket to import into Xray.	`/quality-code:xray ABC-123`
/quality-code:qa	Runs your Xray manual tests with an agent, test by test in the browser, and drafts each defect. You give every verdict; nothing reaches Jira without a preview.	`/quality-code:qa`
/quality-code:seed	Only the test data a ticket needs: seed builder calls (Vitest) or route stubs (Playwright).	`/quality-code:seed ABC-123`

6. Numbers

Command	What it shows	Example
/quality-code:kpi	Tickets gated, fill-ins, Jira comments, pipeline verdicts, and the Quality Program numbers (traceability, completeness, suite honesty, gate integrity, mutation kill rate).	`/quality-code:kpi --since 30d`
/quality-code:consumption	AI usage and estimated cost of the plugin's runs, read locally.	`/quality-code:consumption --by command`

MCP server (every command as a tool)

Mode	How	Notes
Local (stdio)	quality-code-mcp --transport stdio	No sign-in. Install with install.sh, check with --check, --print-config.
Remote (HTTP)	quality-code-mcp --transport http	Entra ID sign-in; only ticket, ticket-eval and tickets-eval; Docker image; open /health.

Remote needs the Entra tenant and client IDs, HTTPS, and a project folder with .claude/quality-code.json.

Backends

Stack	Runs with	Write tests like this
TypeScript, Node	Vitest	describe('ABC-123 …') with one it() per scenario
Python	pytest	class Test… whose docstring starts with ABC-123; plain assert
Java, Kotlin	JUnit 5 or 4 (Maven, Gradle)	@DisplayName("ABC-123 …") on the class
Go	go test	// ABC-123 … above the test function, one t.Run per scenario
C#	dotnet test (xUnit, NUnit, MSTest)	// ABC-123 … above the class
Frontend	Playwright	test.describe('ABC-123 …', { tag: '@ABC-123' })

spec writes them this way for you; the rule matters only when you write tests by hand. Stryker and seeding are Vitest and Playwright only.

Old command → new command

Before	Now
/quality-code:check	/quality-code:run-pipeline --branch (or run-frontend / run-backend)
/quality-code:check spec.ts	/quality-code:run-pipeline spec.ts
/quality-code:check --evals	/quality-code:run-pipeline --evals
/quality-code:start, /quality-code:write KEY	still work; the same as /quality-code:spec